COESIONE ITALIA 21-27 PIEMONTE | Cofinanziato dall'Unione europea | REGIONE PIEMONTE

Cloud Specialist [D68444-15-2025-23]

Unità Formativa : Programmazione Python

Docente: Massimo PAPA

Titolo argomento: Soluzione esercizi sulle classi (OOP)

Intervento realizzato da ITS TECNOLOGIE DELL'INFORMAZIONE E DELLA COMUNICAZIONE

Soluzione esercizi sulle classi (OOP)

Indicazioni sulla consegna **3**
Struttura del codice sorgente 4
Esercizi sulle classi (OOP) **6**
Primo Esercizio 6
Secondo Esercizio 6
Terzo Esercizio 6
Soluzione esercizi sulle classi (OOP) **7**
Primo Esercizio 7
Secondo Esercizio 10
Terzo Esercizio 13

Indicazioni sulla consegna

Implementare gli algoritmi risolutivi dei seguenti esercizi codificandoli in linguaggio python. Partire analizzando il problema seguendo la seguente traccia:

- Quali sono gli input del problema?
- Quali sono gli output?
- Suddividi il problema in problemi più semplici e se lo ritieni opportuno descrivi la soluzione di ogni sottoproblema con un flow-chart
- Andrai a rappresentare ogni algoritmo che risolve un sottoproblema come una funzione
- La funzione restituisce un valore? Se si di che tipo?
- Ogni funzione accetta una lista di parametri formali? Se si di che tipo?
- Esegui la codifica partendo dal programma principale, al suo interno vai a richiamare le funzioni che poi andrai successivamente a definire.
- Continua la codifica dichiarando e definendo tutte le funzioni prima del programma principale che richiama tutte le altre. In testa alle funzioni scrivi un commento che descrive la funzione stessa , cosa restituisce e quali sono i parametri formali.
- Scrivi la sezione dell'inizializzazione variabili scrivendo in un commento il significa della variabile stessa
- Verifica la codifica utilizzando input di test, cercando di provare anche i casi limite.

Struttura del codice sorgente

Relativamente alle indicazioni di scrittura del codice, utilizza il seguente schema generale:

```
'''
    Autore:  Nome Cognome
    Data: gg/mm/aaaa

    Titolo: Testo esercizio

'''

##
## Funzioni:
##


def fun(param1: int, param2: float) -> int:
  '''
     Funzione: fun
      Template per costruire le funzioni
      Parametri formali:
          int Param1 -> descrizione Param1
          float Param2 -> descrizione Param2
      Valore di ritorno:
          int -> descrizione valore di ritorno
  '''
  retValue = 5 # Valore di ritorno della funzione

  return retValue


'''
    Programma principale
    Descrizione sintetica funzionalità
    del programma principale.
'''

# Sezione di input Dati

# Inizializzazioni variabili

# Elaborazione
```

```
# Eventuali sotto processi di Elaborazione

# Sezione di output
```

Esercizi sulle classi (OOP)

Primo Esercizio

Creare una classe Insegnante con attributi nome, età e stipendio, dove stipendio deve essere un attributo privato.

Costruire tutti i metodi getter e setter per gli attributi (anche per quelli pubblici)

Effettuare l'overriding del metodo __str__ in maniera tale che restituisca gli attributi nome e età.

Provare la classe istanziando almeno due oggetti.

Secondo Esercizio

Creare una classe Rettangolo con attributi base e altezza. Costruire tutti i metodi setter e getter per gli attributi. Aggiungere i metodi per il calcolo dell'area e del perimetro.

Implementare un metodo di nome: "contiene" che ha come parametro un oggetto rettangolo. Tale metodo deve restituire true se il rettangolo oggetto chiamante contiene il rettangolo oggetto parametro, false se non lo contiene. Un rettangolo "contiene" un altro quando la sua altezza e la sua base sono maggiori rispettivamente della base e dell'altezza del secondo rettangolo.

Terzo Esercizio

1 - Creare una classe Calcolo con un costruttore di default (senza parametri) che consenta di eseguire vari calcoli su numeri interi.

2 - Creare un metodo chiamato Factorial() che permetta di calcolare il fattoriale di un intero. Testare il metodo istanziando la classe.

3 - Creare un metodo chiamato Sum() che consenta di calcolare la somma dei primi n interi 1 + 2 + 3 + .. + n. Prova questo metodo.

4 - Creare un metodo tableMult() che crei e visualizzi la tabellina di un dato intero. Quindi creare un metodo allTablesMult() per visualizzare tutte le tabelline di numeri interi 1, 2, 3, ..., 9.

Soluzione esercizi sulle classi (OOP)

Primo Esercizio

```
"""
    Autore:  Nome Cognome
    Data: gg/mm/aaaa

    Titolo: Primo eEsercizio
    Creare una classe Insegnante con attributi nome, età
    e stipendio, dove stipendio deve essere un attributo
    privato. Costruire tutti i metodi getter e setter per
    gli attributi (anche quelli pubblici). Effettuare
    l'overriding del metodo __str__ in maniera tale che
    restituisca gli attributi nome e età.
    Provare la classe istanzando due oggetti.

"""


class Insegnante(object):
    """
    Classe Insegnante Es n.1
    La classe modellizza l'entità
    INSEGNANTE. Che ha tre attributi:
      nome: str
      età: int
      stipendio: float

    """

    def __init__(self, nome: str, eta: int, stipendio: float):
        """
        Costruttore dell'oggetto.
        Inizializza i 3 attributi grazie ai parametri
        formali in input al metodo stesso.
        L'attributo stipendio deve essere privato.
        """
        self.nome = nome
        self.eta = eta
        self.__stipendio = stipendio

    def getNome(self) -> str:
```

```
        """
        Getter dell'attributo nome
        """
        return self.nome

    def getEta(self) -> int:
        """
        Getter dell'attributo eta
        """
        return self.eta

    def getStipendio(self) -> float:
        """
        Getter dell'attributo stipendio
        """
        return self.__stipendio

    def setNome(self, val) -> None:
        """
        Setter dell'attributo nome
        """
        self.nome = val

    def setEta(self, val) -> None:
        """
        Setter dell'attributo eta
        """
        self.eta = val

    def setStipendio(self, val) -> None:
        """
        Setter dell'attributo stipendio
        """
        self.__stipendio = val

    def __str__(self):
        return (
            "Nome: "
            + str(self.nome)
            + " Eta': "
            + str(self.getEta()
        )


"""
```

```
Programma principale
Eseguiamo la prova della classe solo se il file.py
e' lanciato come programma. Se fosse importato
le istruzioni all'interno dell'if non vengono eseguite
"""

if __name__ == "__main__":
    ins1 = Insegnante("Paperino", 30, 2000)
    ins2 = Insegnante("Pippo", 43, 1500)

    print(ins1)
    print(ins2)
    ins1.setStipendio(3000)
    print(ins1.getStipendio())
```

Secondo Esercizio

```
"""
    Autore:  Nome Cognome
    Data: gg/mm/aaaa

    Titolo: Secondo esercizio
    Creare una classe Rettangolo con attributi base e altezza.
    Costruire tutti i metodi setter e getter per gli attributi.
    Aggiungere i metodi per il calcolo dell'area e del perimetro.
    Implementare un metodo di nome: "contiene" che ha come
    parametro un oggetto  rettangolo. Tale metodo deve restituire
    true se il rettangolo oggetto chiamante contiene il rettangolo
    oggetto parametro, false se non lo contiene. Un rettangolo
    "contiene" un altro quando la sua altezza e la sua base sono
    maggiori rispettivamente della base e dell'altezza del secondo
    rettangolo.

"""


class Rettangolo(object):
    """
    Classe Rettangolo Es n.2
    La classe modellizza l'entità
    RETTANGOLO. Che ha due attributi:
        base: float
        altezza: float
    """

    def __init__(self, base: float = 1, altezza=1):
        """
        Costruttore dell'oggetto.
        Inizializza i 2 attributi grazie ai parametri
        formali in input al metodo stesso.
        Gli attributi sono privati.
        I parametri del metodo sono tutti predefiniti.
        """
        self.__base = base
        self.__altezza = altezza

    def getBase(self) -> float:
        """
        Getter dell'attributo base
```

```
        """
        return self.__base

    def getAltezza(self) -> float:
        """
        Getter dell'attributo altezza
        """
        return self.__altezza

    def setBase(self, val) -> None:
        """
        Setter dell'attributo base
        """
        self.__base = val

    def setAltezza(self, val) -> None:
        """
        Getter dell'attributo altezza
        """
        self.__altezza = val

    def getArea(self) -> float:
        """
        Metodo: getArea
        Metodo che restituisce l'area
        del rettangolo.
        Valore di ritorno:
        float -> area
        """
        return self.__base * self.__altezza

    def getPerimetro(self):
        """
        Metodo: getPerimetro
        Metodo che restituisce il perimetro
        del rettangolo.
        Valore di ritorno:
        float -> perimentro
        """
        return 2 * self.__base + 2 * self.__altezza

    def __str__(self):
        return "Base: " + str(self.__base) + " altezza: " +
str(self.__altezza)

    def contiene(self, rettangolo) -> bool:
```

```
        """
        Metodo: contiene
        Metodo che verifica se il rettangolo
        contiene un rettangolo passato come
        parametro al metodo.
        Valore di ritorno:
        bool -> True se contiene altrimenti False
        """
        ret = False
        if (
            rettangolo.getBase() < self.__base
            and rettangolo.getAltezza() < self.__altezza
        ):
            ret = True
        return ret


"""
Programma principale
Eseguiamo la prova della classe solo se il file.py
e' lanciato come programma. Se fosse importato
le istruzioni all'interno dell'if non vengono eseguite
"""

if __name__ == "__main__":
    rect1 = Rettangolo(2, 2)
    rect2 = Rettangolo()

    print(rect1)
    print(rect2)
    print(rect1.contiene(rect2))
    print(rect2.contiene(rect1))
```

Terzo Esercizio

```
"""
    Autore:  Nome Cognome
    Data: gg/mm/aaaa

    Titolo: Terzo esercizio
    1 - Creare una classe Calcolo con un costruttore di default
    (senza parametri) che consenta di eseguire vari calcoli su
    numeri interi.
    2 - Creare un metodo chiamato Factorial() che permetta di
    calcolare il fattoriale di un intero. Testare il metodo
    istanziando la classe.
    3 - Creare un metodo chiamato Sum() che consenta di calcolare
    la somma dei primi n interi 1 + 2 + 3 + .. + n. Prova questo metodo.
    4 - Creare un metodo tableMult() che crea e visualizza la tabellina
    di un dato intero. Quindi creare un metodo allTablesMult() per
    visualizzare tutte le tabelline di numeri interi 1, 2, 3, ..., 9.

"""

class Calcolo(object):
    """
    Classe Calcolo Es n.3
    La classe modellizza l'entità
    CALCOLO. Ha un attributo privato: numero.

    """

    def __init__(self):
        '''
        Metodo costruttore della classe
        Definisce l'attributo privato numero
        iniziandolo a 1
        parametri formali: nessuno

        '''
        self.__numero=0

    def getNumero(self) -> int:
        """
        Metodo getter per l'attributo
        privato numero
        """
```

```
        return self.__numero

    def setNumero(self,val:int):
        """
        Metodo setter per l'attributo
        privato numero
        """
        self.__numero=val

    def factorial(self) -> int:
        """
        Metodo: factorial
        Metodo che restituisce il
        fattoriale del numero
        passato come parametro.
        Parametri formali:
        Nessuno
        Valore di ritorno:
        int -> fattoriale
        bool -> True se errore altrimenti False
        """
        # Inizializzo errore
        err = True
        if self.__numero >= 0:
            # Inizializzo fattoriale
            fatt = 1
            for x in range(1,self.__numero + 1):
                fatt *= x
            err = False
        return fatt, err

    def sum(self) -> int:
        """
        Metodo: sum
        Metodo che restituisce la
        somma dei primi num numeri
        naturali.
        Parametri formali:
        nessuno
        Valore di ritorno:
        bool -> True se errore altrimenti False
        int -> somma
        """
        err = True
        somma = 0
```

```
        if self.__numero >= 0:
            for x in range(self.__numero + 1):
                somma += x
            err = False
        return somma, err

    def __tableMult(self, num: int) -> int:
        """
        Metodo: tableMult
        Metodo che restituisce la
        tabellina del numero num
        Parametri formali:
        int -> num
        Valore di ritorno:
        str -> tabellina di num
        """
        # Inizializzazione strnga di ritorno
        ret = "\n"
        for x in range(1, 11):
            ret += str(x * num) + " "
        return ret

    def allTableMult(self):
        """
        Metodo: allTableMult
        Metodo che restituisce la
        tabellina dei numeri da 1 a 9
        Parametri formali:
        None
        Valore di ritorno:
        str -> tabellina
        """
        ret = ""
        for x in range(1, 10):
            ret += self.__tableMult(x)
        return ret


"""
Programma principale
Eseguiamo la prova della classe solo se il file.py
e' lanciato come programma. Se fosse importato
le istruzioni all'interno dell'if non vengono eseguite
"""
if __name__ == "__main__":
```

```
calc = Calcolo()
print(calc.factorial())
print(calc.sum())
print(calc.allTableMult())

calc.setNumero(3)
print(calc.factorial())
```